CRE DES

Candidate Task

backend dev intern

thinking beyond limits

Task Assignment Date : 25th April, 2026

credes.in

Task Brief — Backend Intern

Internship Track: Backend Engineering — Node.js / API Systems **Organization:** Credes TechLabs Private Limited **Assignment Type:** Multi-Platform AI Content Publishing Engine **Submission Deadline:** 4 days from receipt of this brief (EOD on Day 4)

The Product Context

You are building the backend engine for **Postly** — a multi-platform AI content publishing system. A user drops a raw idea into a Telegram bot (or WhatsApp), picks their target platforms and AI model, and the system generates platform-specific content and posts it automatically — no dashboard needed for the core publish flow.

Your job is the entire server-side: auth, user onboarding, social account management, the Telegram bot, the AI content engine, the publishing pipeline, and the REST API that a frontend will consume.

> This is not a toy API. We want to see how you think about real systems — schema design, queue architecture, failure handling, and integration depth. Clean code and defensible decisions beat clever hacks.

What You Are Building

System Overview

```
User → Telegram Bot → Your API Server → AI Engine (OpenAI / Claude)
                                      → Publishing Queue (BullMQ + Redis)
                                      → Platform APIs (Twitter, LinkedIn, etc.)
                                      ↓
                                PostgreSQL (users, posts, accounts, jobs)
```

Required Tech Stack

Layer	Technology
Runtime	Node.js (v18+)
Framework	Express.js (or Fastify — justify if Fastify)
Database	PostgreSQL (primary), Redis (queues + sessions)
ORM / Query Builder	Prisma or Knex (your choice, justify)
Bot Interface	Telegram Bot API (via `node-telegram-bot-api` or `grammy`)
AI Integration	OpenAI API + Anthropic Claude API (both required)
Queue	BullMQ (backed by Redis)
Auth	JWT (access + refresh token pattern)
Deployment	Railway / Render / fly.io / any cloud (live URL required)
Containerisation	Docker + docker-compose (for local dev)
Testing	Jest + Supertest (min 5 tests)

Core Requirements

1. User Auth & Onboarding

Build a complete auth system with the following endpoints:

```
POST   /api/auth/register          - email, password, name
POST   /api/auth/login             - returns access_token + refresh_token
POST   /api/auth/refresh           - refresh token rotation
POST   /api/auth/logout            - invalidate refresh token
GET    /api/auth/me                - current user profile
```

Requirements:

- Passwords hashed with bcrypt (min cost 12)
- Access token: short-lived (15 min), signed JWT
- Refresh token: long-lived (7 days), stored in DB (not just stateless), rotated on use
- All protected routes must validate JWT via middleware — no exceptions

---

2. User Profile & Social Account Setup

```
PUT    /api/user/profile           - update name, bio, default tone, default language
GET    /api/user/profile           - fetch profile

POST   /api/user/social-accounts   - add a social account (platform + credentials/tokens)
GET    /api/user/social-accounts   - list connected accounts
DELETE /api/user/social-accounts/:id - disconnect an account

PUT    /api/user/ai-keys           - store encrypted OpenAI and/or Anthropic API keys
```

Social account linking:

- Store OAuth tokens (access + refresh) per platform per user, encrypted at rest (AES-256 or equivalent)
- Supported platforms: Twitter/X, LinkedIn, Instagram, Threads
- OAuth flow scaffolding is acceptable — full OAuth callback implementation is a bonus

Schema hint (PostgreSQL):

```
users (id, email, password_hash, name, bio, default_tone, default_language, created_at)
social_accounts (id, user_id, platform, access_token_enc, refresh_token_enc, handle,
connected_at)
ai_keys (id, user_id, openai_key_enc, anthropic_key_enc, updated_at)
```

---

3. Telegram Bot — Conversational Publishing Flow

Build a Telegram bot with a stateful, multi-step conversation flow. The bot is the primary interface for publishing content.

Bot conversation flow:

```
User: /start or /post

Bot:  "Hey [name] 👋 What type of post is this?"
      [Options: Announcement | Thread | Story | Promotional | Educational | Opinion]

User: picks type

Bot:  "Which platforms should I post to?"
      [Options: Twitter/X | LinkedIn | Instagram | Threads | All]
      (multi-select - user can pick multiple)

User: picks platforms

Bot:  "What tone should the content have?"
      [Options: Professional | Casual | Witty | Authoritative | Friendly]
```

```
User: picks tone

Bot:  "Which AI model do you want to use?"
      [Options: GPT-4o (OpenAI) | Claude Sonnet (Anthropic)]

User: picks model

Bot:  "Tell me the idea or core message — keep it brief."

User: types idea (free text, max 500 chars)

Bot:  "Generating your content... ⚙️"
      → calls AI engine
      → returns preview per platform:

      📱 Twitter/X (280 chars):
      "..."

      💼 LinkedIn (1200 chars):
      "..."

      📸 Instagram (caption):
      "..."

      Confirm and post? [Yes, Post Now ✅] [Edit Idea ✏️] [Cancel ❌]

User: confirms

Bot:  Posts to each platform via publishing queue
      Returns status per platform with success/failure message
```

Technical requirements:

- Store conversation state in Redis (keyed by Telegram chat ID) — conversations expire after 30 min of inactivity
- Webhook-based bot (not polling) in production
- `/status` command — shows last 5 posts and their platform statuses
- `/accounts` command — shows connected social accounts
- `/help` command — lists all commands
- Handle edge cases: user sends unexpected input, session timeout, API failure

---

4. AI Content Engine

```
POST   /api/content/generate       — internal + bot-triggered
```

Request body:

```
{
  "idea": "string (max 500 chars)",
  "post_type": "announcement | thread | story | promotional | educational | opinion",
  "platforms": ["twitter", "linkedin", "instagram", "threads"],
  "tone": "professional | casual | witty | authoritative | friendly",
  "language": "en | hi | ar | (extensible)",
  "model": "openai | anthropic"
}
```

Response:

```
{
  "generated": {
    "twitter": { "content": "...", "char_count": 240, "hashtags": ["#tag1"] },
    "linkedin": { "content": "...", "char_count": 1100 },
    "instagram": { "content": "...", "hashtags": ["#tag1", "#tag2"] },
    "threads": { "content": "..." }
  },
  "model_used": "claude-sonnet-4-5",
  "tokens_used": 820
}
```

Platform-specific generation rules you must enforce:

- Twitter/X: max 280 characters, 2–3 hashtags, punchy opener
- LinkedIn: 800–1300 characters, professional tone regardless of global tone setting, 3–5 hashtags
- Instagram: caption + 10–15 hashtags, emoji-friendly
- Threads: 500 characters max, conversational

AI prompt architecture:

- Use a system prompt that enforces platform constraints
- User's tone and language preference must be injected into the prompt
- Use the user's own API key (from `ai_keys`) if available; fall back to a platform key
- Both OpenAI and Anthropic integrations must be working — not mocked

---

5. Publishing Engine

```
POST   /api/posts/publish          - queue a post for immediate or scheduled publishing
POST   /api/posts/schedule         - queue a post with a future publish_at timestamp
GET    /api/posts                  - list posts with status per platform (paginated)
GET    /api/posts/:id              - single post detail
```

```
POST   /api/posts/:id/retry        - retry failed platform jobs
DELETE /api/posts/:id              - cancel a scheduled post (if not yet published)
```

Queue architecture (BullMQ):

- One job per platform per post (Twitter job, LinkedIn job, etc.) — not one monolithic job
- Jobs include: payload, platform, user_id, retry count
- Retry policy: 3 attempts with exponential backoff (1s → 5s → 25s)
- Failed jobs must be recorded in DB with error reason
- Job status: `queued | processing | published | failed | cancelled`

Post schema:

```
posts (id, user_id, idea, post_type, tone, language, model_used, created_at, publish_at,
status)
platform_posts (id, post_id, platform, content, status, published_at, error_message,
attempts)
```

---

6. REST API for Dashboard (Frontend Consumption)

Expose these additional endpoints so the frontend can build a dashboard:

```
GET    /api/dashboard/stats        - total posts, success rate, posts per platform
GET    /api/posts?page=1&limit=10&status=published - paginated post history
GET    /api/posts/:id              - post detail with per-platform status
```

All list endpoints must support:

- Pagination (`page`, `limit`)
- Filtering by `status`, `platform`, `date_range`
- Consistent response envelope: `{ data, meta: { total, page, limit }, error }`

⚠️ Non-Negotiable Submission Standards

> **Read this section before writing a single line of code. Failing any point here means automatic non-selection — regardless of how good the code is.**

GitHub — Source Code & Commit History

Your entire project must live on a public GitHub repository. This is not optional. There is no alternative submission method.

We do not just review your final code. **We review your commit history.** Your commits are a window into how you actually work — and we take this seriously.

What we expect from your commit history:

- Commits are made incrementally as you build — not in one or two giant dumps at the end
- Each commit message is meaningful and describes what actually changed (e.g. `feat: add JWT refresh token rotation`, `fix: handle Redis session timeout in bot flow`, `chore: add docker-compose for local dev`)
- The history should show a logical build progression: schema → auth → bot → AI engine → publishing → tests → deployment
- Branch usage is a plus (e.g. `feature/telegram-bot`, `feature/ai-engine`) but a clean main branch with good commits is acceptable

Commit history red flags that will count against you:

- A single commit with everything in it
- Commits like "final", "done", "fix stuff", "update"
- Committing `.env` files, API keys, or secrets at any point (even if later removed — git history is permanent)
- Copying in large amounts of boilerplate in one commit with no follow-up refinement

Live Deployment — Usable API Required

Your API must be deployed and publicly accessible at a live URL. We will call your endpoints directly to verify functionality — not just read your code.

- Deploy to Railway, Render, fly.io, or any equivalent cloud platform
- The Telegram bot must be live and responding (webhook configured, not polling)
- Include the base URL prominently at the top of your `README.md`
- Include a working Postman / Bruno collection that points to your live URL
- The deployment must remain live for at least 7 days after your submission

If your API is not deployed and reachable at submission time, your task will not be reviewed.

On AI-Assisted Development

AI tools (Copilot, Cursor, Claude, ChatGPT) are permitted and encouraged — we hire for this skill. However:

> **Complete reliance on AI-generated code with no genuine understanding will result in immediate non-selection.**

We can identify fully AI-generated codebases. During review, we will ask you to explain specific implementation decisions line by line. If you cannot, it counts against you — regardless of what's in the repo.

Use AI as a force multiplier on your own understanding. Not as a replacement for it.

Submission Requirements

Provide a single public GitHub repository containing:

Must Include

`README.md` covering:

- Local setup (clone → docker-compose up → working)
- All environment variables with descriptions (use `.env.example`)
- API documentation (or link to Postman collection)
- Telegram bot setup instructions (BotFather token, webhook URL)
- Architecture overview with a data flow diagram (ASCII or image)
- Design decisions and trade-offs you made
- Known issues and limitations

`ARCHITECTURE.md` covering:

- How a post flows from Telegram bot → AI engine → queue → platform API
- How conversation state is managed in Redis
- Schema design decisions and indexing strategy
- How you handle partial failures (one platform fails, others succeed)

`AI_USAGE.md` — required (see AI Policy below)

Working `.env.example` — every secret the app needs, documented

`docker-compose.yml` — spins up the app, PostgreSQL, and Redis locally with one command

Postman / Bruno collection — all endpoints importable and testable

Tests — minimum 5, covering:

- Auth middleware (valid token, expired token, missing token)
- Content generation input validation
- Queue job creation on publish
- Post status retrieval
- At least one integration test hitting the DB

Live deployment URL — non-negotiable. The API must be deployed and accessible. Include base URL in README.

2-minute Loom / screen recording showing:

- Telegram bot end-to-end flow (idea → generate → confirm → post)
- At least one successful platform post
- API responses in Postman or terminal

Nice-to-Have (Boosts Score Significantly)

- WhatsApp bot via Twilio / 360dialog (same flow as Telegram)
- Full OAuth callback implementation for Twitter and LinkedIn (not just token storage)
- Rate limiting per user (express-rate-limit or custom Redis counter)
- Webhook signature verification for Telegram
- Scheduled posts with cron-based job dispatch
- `/api/posts/:id/analytics` — fetch engagement stats from platform APIs
- Language detection on the idea input and auto-setting language
- Soft-delete pattern on posts with restore capability

AI Usage Policy (Credes)

You must include `AI_USAGE.md` in your repo.

Permitted: Using AI tools (Copilot, Cursor, Claude, ChatGPT) to assist your development — scaffolding, debugging, code review, test generation, documentation. This is an expected and

valued skill.

Not permitted: Submitting a codebase you do not understand. Fully AI-generated submissions — where the candidate cannot explain what the code does or why decisions were made — will be marked for non-selection. We have reviewed enough AI-generated code to identify it quickly.

What good AI-assisted development looks like: You use AI to move faster on things you already understand. You validate, refactor, and own everything that ends up in your repo. Your commit history shows real iteration — not a paste of generated output.

Required in `AI_USAGE.md`: For every significant section where you used AI assistance, document: what tool, what prompt or task, and what you changed or validated. This transparency is a plus — it shows maturity in how you use these tools.

Evaluation Criteria

Criterion	Weight	What We Look For
API Design & Correctness	18%	Clean REST conventions, consistent response envelopes, proper HTTP status codes, no leaking internals
Telegram Bot — Flow & Reliability	18%	Stateful conversation works end-to-end, handles bad input, session management in Redis is correct
AI Integration Depth	14%	Both models integrated, platform-specific prompts enforced, token/cost awareness, graceful API failure handling
Queue & Publishing Architecture	14%	Per-platform jobs, retry with backoff, partial failure handling, job status accurately tracked in DB
GitHub — Commit Quality & History	12%	Commits are incremental and meaningful, messages describe real changes, history shows logical build progression, no secrets ever committed
Auth & Security	8%	Token rotation, encrypted secrets at rest, no plaintext credentials anywhere in code or logs
Schema Design & DB	8%	Normalized schema, appropriate indexes, migrations in version control, no N+1 patterns
Code Quality & Structure	4%	Layered architecture (routes → controllers → services → DB), no business logic in route handlers
Testing	4%	Tests are meaningful, not just coverage padding — do they catch real bugs?

> A fully working Telegram bot that posts real content to at least one real platform will score higher than a perfectly structured API that never connects to anything real.
>
> A clean, incremental commit history showing how you think and build will score higher than a single polished commit dropped at the end.